

顺时 × SEASONS

自动化开发编排系统

一键运行 · 自动完成全部代码生成
3套方案 · Claude Code / API脚本 / Cursor

V1.0

一键运行，自动完成全部项目代码生成

方案一：Claude Code 一键执行方案（推荐）

原理

Claude Code 是命令行AI编程工具，支持读取本地文件、创建文件、执行命令。你只需要把以下内容保存为 `CLAUDE.md` 文件放在项目根目录，然后运行 `claude` 命令，它会自动按顺序完成所有开发。

使用步骤

```
# 1. 安装 Claude Code
npm install -g @anthropic-ai/claude-code

# 2. 创建项目目录
mkdir shunshi && cd shunshi

# 3. 把下面的 CLAUDE.md 文件放到项目根目录

# 4. 把终极开发文档放到项目根目录
#   - 顺时_终极开发文档.md
#   - AI编程工具开发执行手册.md

# 5. 启动 Claude Code
claude

# 6. 输入以下指令：
#   "请阅读 CLAUDE.md 文件，按照里面的指令从Phase 0开始，逐步完成整个项目的开发。每完成一个Phase就告诉我进度，然后自动继续下一个Phase。"
```

CLAUDE.md 文件内容（保存到项目根目录）

```
# 顺时项目 Claude Code 自动化开发指令

## 你的角色
你是顺时项目的首席架构师和全栈开发工程师。你需要从零开始构建整个项目。
请严格按照本文件的Phase顺序执行，每个Phase的每个步骤都必须完成后在进入下一步。

## 项目概述
顺时是一款AI养生生活陪伴系统。技术栈：
- 后端：Python 3.11 + FastAPI + SQLAlchemy 2.0 + Celery + Redis + PostgreSQL 16
- 前端：React Native 0.73 + Zustand + TypeScript
- AI：通义千问(主) + OpenAI(备) + pgvector(RAG)

## 核心设计规范
详见同目录下的 `顺时_终极开发文档.md`（必须先读取该文件）。

## 开发规范
1. Python使用类型注解、async/await、Pydantic v2
2. 命名：Python用snake_case，TypeScript用camelCase
3. 所有ID用UUID
4. API响应格式：{"code": 0, "message": "success", "data": {...}}
5. 错误响应：{"code": "ERROR_CODE", "message": "中文友好消息", "status": 400}
6. 所有错误消息用温暖的中文
7. 每个文件包含完整import和类型注解
8. 每个函数包含docstring
9. 包含完整错误处理

## 执行顺序

### Phase 0：项目骨架
按以下顺序创建文件，每创建一个就测试（语法检查）：

0.1 创建 `backend/requirements.txt`（全部依赖）
0.2 创建 `backend/app/__init__.py`
0.3 创建 `backend/app/config.py`（pydantic-settings，所有环境变量）
0.4 创建 `backend/app/database.py`（异步SQLAlchemy引擎 + Redis + get_db）
0.5 创建 `backend/app/core/__init__.py`
0.6 创建 `backend/app/core/exceptions.py`（自定义异常，中文消息）
0.7 创建 `backend/app/core/response.py`（统一响应函数）
0.8 创建 `backend/app/core/auth.py`（JWT创建/验证/刷新）
0.9 创建 `backend/app/core/dependencies.py`（get_current_user等依赖）
0.10 创建 `backend/app/core/rate_limiter.py`（Redis频率限制）
0.11 创建 `backend/app/main.py`（FastAPI入口，CORS，异常处理器，/health）
0.12 创建 `backend/Dockerfile`
0.13 创建 `docker-compose.yml`（api + celery + db + redis + meilisearch）
0.14 创建 `backend/.env.example`
```

Phase 1: 数据库模型

严格按照终极开发文档第三部分的SQL设计创建。每创建一个Model同时创建对应的Schema。

```
1.1 `backend/app/models/__init__.py` + `backend/app/models/base.py` (Base类)
1.2 `backend/app/models/user.py` + `backend/app/schemas/user.py`
1.3 `backend/app/models/solar_term.py` + `backend/app/schemas/solar_term.py`
1.4 `backend/app/models/constitution.py` + `backend/app/schemas/constitution.py`
1.5 `backend/app/models/recipe.py` + `backend/app/schemas/recipe.py`
1.6 `backend/app/models/tea.py` + `backend/app/schemas/tea.py`
1.7 `backend/app/models/acupoint.py` + `backend/app/schemas/acupoint.py`
1.8 `backend/app/models/exercise.py` + `backend/app/schemas/exercise.py`
1.9 `backend/app/models/article.py` + `backend/app/schemas/article.py`
1.10 `backend/app/models/audio.py` + `backend/app/schemas/audio.py`
1.11 `backend/app/models/chat.py` + `backend/app/schemas/chat.py`
1.12 `backend/app/models/journal.py` + `backend/app/schemas/journal.py`
1.13 `backend/app/models/membership.py` + `backend/app/schemas/membership.py`
1.14 `backend/app/models/community.py` + `backend/app/schemas/community.py`
1.15 `backend/app/models/notification.py` + `backend/app/schemas/notification.py`
1.16 `backend/app/models/analytics.py`
1.17 `backend/app/models/banner.py` + `backend/app/schemas/banner.py`
1.18 `backend/app/models/knowledge.py` (pgvector向量表)
1.19 更新 `models/__init__.py` 导入全部模型
1.20 配置Alembic: `backend/alembic.ini` + `backend/migrations/env.py`
1.21 生成并运行初始迁移
```

Phase 2: Service层

每个Service文件包含完整的业务逻辑实现。参照终极开发文档第五部分。

```
2.1 `backend/app/services/solar_term_engine.py`
    函数: get_current_solar_term, get_upcoming, get_calendar, get_by_code, get_recommendations, check_change
2.2 `backend/app/services/constitution_scorer.py`
    函数: get_questions, calculate_scores(王琦标准算法), submit_test, generate_ai_report
2.3 `backend/app/services/ai_companion.py`
    函数: build_system_prompt, retrieve_knowledge(RAG), chat_stream(SSE), parse_cards
2.4 `backend/app/services/rag_retriever.py`
    函数: embed_text, search_similar, retrieve_context
2.5 `backend/app/services/recommendation.py`
    函数: get_personalized_recommendations(4维加权), get_home_data
2.6 `backend/app/services/journal_analyzer.py`
    函数: analyze_journal(AI), calculate_sleep_score, analyze_mood_trends
2.7 `backend/app/services/report_generator.py`
    函数: generate_weekly_report, generate_monthly_report
2.8 `backend/app/services/notification_engine.py`
    函数: send_push, schedule_reminders, send_solar_term_notification
2.9 `backend/app/services/payment_service.py`
    函数: create_wechat_order, verify_wechat, create_alipay_order, verify_apple_iap
2.10 `backend/app/services/share_card.py`
    函数: generate_constitution_card, generate_checkin_card, generate_report_card
2.11 `backend/app/services/cache_manager.py`
    函数: get_cached, set_cached, invalidate, warm_cache
2.12 `backend/app/services/search_service.py`
    函数: index_content, search(Meilisearch), unified_search
```

Phase 3: API路由

每个路由文件只做参数校验→调Service→返回响应。

```
3.1 `backend/app/api/__init__.py` + `backend/app/api/v1/__init__.py` (路由注册)
3.2 `backend/app/api/v1/auth.py` - POST sms/send, sms/verify, wechat, apple, refresh, logout
3.3 `backend/app/api/v1/user.py` - GET/PUT profile, settings, family CRUD
3.4 `backend/app/api/v1/home.py` - GET /home 聚合接口
3.5 `backend/app/api/v1/solar_term.py` - current, upcoming, calendar, {code}, recipes/teas/acupoints
3.6 `backend/app/api/v1/constitution.py` - questions, test, result, types, plan
3.7 `backend/app/api/v1/recipe.py` - list(筛选), detail, recommended
3.8 `backend/app/api/v1/tea.py` - list, detail, recommended
3.9 `backend/app/api/v1/acupoint.py` - list(按症状/体质), detail
3.10 `backend/app/api/v1/exercise.py` - list, detail
3.11 `backend/app/api/v1/article.py` - list, detail + `audio.py` - list, detail, categories
3.12 `backend/app/api/v1/chat.py` - sessions CRUD, messages(SSE StreamingResponse)
3.13 `backend/app/api/v1/journal.py` - today, create/update, history, calendar, stats
3.14 `backend/app/api/v1/report.py` - weekly, monthly, generate
3.15 `backend/app/api/v1/membership.py` - plans, subscribe, verify, status, cancel
3.16 `backend/app/api/v1/payment.py` - wechat/alipay/apple webhooks
3.17 `backend/app/api/v1/search.py` - unified search
3.18 `backend/app/api/v1/community.py` - posts CRUD, like, comment
3.19 `backend/app/api/v1/share.py` - generate-card, referral-code
3.20 `backend/app/api/v1/notification.py` - reminders, notifications
3.21 更新 `main.py` 注册全部路由
```

Phase 4: Celery异步任务

```
4.1 `backend/app/celery_config.py` (Celery实例配置)
4.2 `backend/app/tasks/__init__.py`
4.3 `backend/app/tasks/solar_term_check.py` (每日0:05检查节气变化)
4.4 `backend/app/tasks/notification_scheduler.py` (每分钟检查定时提醒)
4.5 `backend/app/tasks/report_tasks.py` (周一生成周报, 月初生成月报)
4.6 `backend/app/tasks/journal_analysis.py` (用户提交日记后AI分析)
4.7 `backend/app/tasks/cache_warming.py` (每小时预热缓存)
4.8 `backend/app/tasks/data_cleanup.py` (每日清理过期数据)
```

Phase 5: 知识库数据

```
5.1 创建 `backend/app/knowledge/solar_terms.json` (24个节气完整养生数据)
5.2 创建 `backend/app/knowledge/constitutions.json` (9种体质完整数据)
5.3 创建 `backend/app/knowledge/constitution_questions.json` (33题标准量表)
5.4 创建 `backend/app/knowledge/recipes.json` (至少50个初始食谱)
5.5 创建 `backend/app/knowledge/teas.json` (至少30个茶饮)
5.6 创建 `backend/app/knowledge/acupoints.json` (至少20个常用穴位)
```

```
5.7 创建 `backend/app/knowledge/exercises.json` (八段锦8式 + 办公室操)
5.8 创建 `backend/app/knowledge/shichen.json` (12时辰养生)
5.9 创建 `backend/app/knowledge/food_therapy.json` (10种常见症状食疗方)
5.10 创建 `backend/app/knowledge/tcm_quotes.json` (20条经典名言)
5.11 创建 `backend/scripts/seed_data.py` (读取JSON导入数据库的脚本)
5.12 创建 `backend/scripts/build_embeddings.py` (知识库向量化脚本)

### Phase 6: 前端基础框架

6.1 初始化React Native项目: `npx react-native init ShunShi --template react-native-template-typescript`
6.2 安装依赖: zustand, @react-navigation/native, react-native-reanimated, lottie-react-native, victory-native, react-native-mmkv, axios, @
6.3 创建 `mobile/src/theme/colors.ts` (品牌色系统)
6.4 创建 `mobile/src/theme/spacing.ts` (间距系统)
6.5 创建 `mobile/src/theme/typography.ts` (字体系统)
6.6 创建 `mobile/src/components/ui/Button.tsx` (primary/secondary/ghost)
6.7 创建 `mobile/src/components/ui/Card.tsx`
6.8 创建 `mobile/src/components/ui/Input.tsx`
6.9 创建 `mobile/src/components/ui/Tag.tsx`
6.10 创建 `mobile/src/components/ui/Loading.tsx` (骨架屏)
6.11 创建 `mobile/src/services/api.ts` (Axios实例+拦截器+Token刷新)
6.12 创建 `mobile/src/stores/authStore.ts` (登录状态管理)
6.13 创建 `mobile/src/stores/userStore.ts` (用户信息)
6.14 创建 `mobile/src/stores/appStore.ts` (节气/主题/设置)
6.15 创建 `mobile/src/navigation/RootNavigator.tsx` (Auth/Onboarding/Main分流)
6.16 创建 `mobile/src/navigation/TabNavigator.tsx` (5个Tab)

### Phase 7: 前端页面

7.1 `mobile/src/screens/auth/LoginScreen.tsx` (手机号+验证码登录)
7.2 `mobile/src/screens/onboarding/` 完整引导流程 (Welcome-Quiz-Preferences-Complete)
7.3 `mobile/src/screens/home/HomeScreen.tsx` (首页完整布局—10大区块)
7.4 `mobile/src/screens/home/components/` (SolarTermCard, DailyTips, CheckinBar, QuickActions, RecipeCarousel)
7.5 `mobile/src/screens/wellness/WellnessScreen.tsx` (Tab切换页)
7.6 `mobile/src/screens/wellness/SolarTermDetail.tsx`
7.7 `mobile/src/screens/recipe/RecipeListScreen.tsx` + `RecipeDetailScreen.tsx`
7.8 `mobile/src/screens/recipe/components/` (FilterBar, RecipeCard, IngredientList, StepList)
7.9 `mobile/src/screens/tea/TeaListScreen.tsx` + `TeaDetailScreen.tsx`
7.10 `mobile/src/screens/acupoint/AcupointListScreen.tsx` + `AcupointDetailScreen.tsx`
7.11 `mobile/src/screens/exercise/ExerciseListScreen.tsx` + `PracticeScreen.tsx` (跟练模式)
7.12 `mobile/src/screens/chat/ChatScreen.tsx` (AI对话, SSE流式接收, 推荐卡片渲染)
7.13 `mobile/src/screens/journal/JournalScreen.tsx` (日历+日记入口)
7.14 `mobile/src/screens/journal/JournalEntryScreen.tsx` (5模块日记录入)
7.15 `mobile/src/screens/journal/ReportScreen.tsx` (周报/月报展示+图表)
7.16 `mobile/src/screens/profile/ProfileScreen.tsx` (个人中心)
7.17 `mobile/src/screens/profile/MembershipScreen.tsx` (会员)
7.18 `mobile/src/screens/profile/FamilyScreen.tsx` (家庭养生)
7.19 `mobile/src/screens/profile/SettingsScreen.tsx`
7.20 `mobile/src/screens/sleep/SleepScreen.tsx` (助眠音频+呼吸练习)
7.21 `mobile/src/components/player/AudioPlayer.tsx` (音频播放器迷你栏+全屏)
7.22 `mobile/src/components/breathing/BreathingExercise.tsx` (4-7-8动画)

### Phase 8: 测试与优化

8.1 创建 `backend/tests/conftest.py` (测试配置, 测试数据库)
8.2 创建 `backend/tests/test_auth.py`
8.3 创建 `backend/tests/test_solar_term.py`
8.4 创建 `backend/tests/test_constitution.py`
8.5 创建 `backend/tests/test_recipe.py`
8.6 创建 `backend/tests/test_chat.py`
8.7 创建 `backend/tests/test_journal.py`
8.8 运行全部测试, 修复失败项

### Phase 9: 部署配置

9.1 创建 `docker-compose.prod.yml` (生产环境Docker配置)
9.2 创建 `.github/workflows/backend-ci.yml` (后端CI)
9.3 创建 `.github/workflows/deploy.yml` (自动部署)
9.4 创建 `backend/scripts/deploy.sh` (部署脚本)
9.5 创建 `README.md` (项目说明+启动指南)

## 执行规则

1. **严格按Phase顺序执行**，不要跳步
2. **每创建一个文件后立即验证** (检查import是否正确、语法是否通过)
3. **如果某一步有依赖未满足**，先完成依赖
4. **所有代码必须完整**，不要用"...省略"
5. **Python代码运行 `python -c "import backend.app.models.user"` 验证**
6. **完成每个Phase后报告进度**

## 现在开始执行 Phase 0。
```

方案二：自动化脚本编排方案（适用于API调用）

如果你使用GLM API或其他支持API调用的模型，可以用以下Python脚本自动化执行81个开发任务。

安装

```
pip install openai zhipuai httpx
```

自动化编排脚本

保存为 `auto_dev.py`：

```
#!/usr/bin/env python3
"""
顺时项目自动化开发编排脚本
用法: python auto_dev.py --api-key YOUR_KEY --model glm-4-plus
"""

import os
import sys
import json
import time
import argparse
import argparse
from pathlib import Path
from zhipuai import ZhipuAI # 如果用GLM

# ===== 配置 =====
PROJECT_ROOT = Path("./shunshi")
BACKEND_DIR = PROJECT_ROOT / "backend" / "app"

# ===== 项目记忆 =====
PROJECT_CONTEXT = ""
# 顺时项目上下文（每次注入）

## 项目
顺时 — AI养生生活陪伴系统, Python 3.11 + FastAPI + SQLAlchemy 2.0 + PostgreSQL 16 + Redis + Celery

## 代码规范
- 类型注解、async/await、Pydantic v2
- snake_case (Python), camelCase (TypeScript)
- UUID主键, JSONB复杂字段
- API响应: {"code": 0, "message": "success", "data": {...}}
- 错误响应: {"code": "ERROR_CODE", "message": "中文温暖消息", "status": 400}
- 所有文件完整, 不要省略代码

## 已完成文件
{completed_files}
"""

# ===== 任务定义 =====
TASKS = [
    # Phase 0: 骨架
    {
        "id": "0.1",
        "file": "backend/requirements.txt",
        "prompt": """创建 requirements.txt, 包含以下精确版本的依赖:
fastapi==0.109.0, uvicorn[standard]==0.27.0, sqlalchemy==2.0.25, alembic==1.13.1,
asyncpg==0.29.0, psycpg2-binary==2.9.9, redis==5.0.1, celery==5.3.6,
pydantic==2.5.3, pydantic-settings==2.1.0, python-jose[cryptography]==3.3.0,
passlib[bcrypt]==1.7.4, python-multipart==0.0.6, httpx==0.26.0, pillow==10.2.0,
meilisearch==0.31.0, openai==1.12.0, dashscope==1.14.1, oss2==2.18.4,
pgvector==0.2.4, numpy==1.26.3

每行一个包名==版本号, 不要有其他内容。""",
    },
    {
        "id": "0.2",
        "file": "backend/app/config.py",
        "prompt": """创建 config.py, 使用pydantic-settings:

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
 # 数据库
 DATABASE_URL: str = "postgresql+asyncpg://postgres:password@localhost:5432/shunshi"
 REDIS_URL: str = "redis://localhost:6379/0"

 # JWT
 JWT_SECRET_KEY: str = "your-secret-key"
 JWT_ALGORITHM: str = "HS256"
 JWT_ACCESS_TOKEN_EXPIRE_MINUTES: int = 1440
 JWT_REFRESH_TOKEN_EXPIRE_DAYS: int = 30

 # AI
 QWEN_API_KEY: str = ""
 OPENAI_API_KEY: str = ""
 AI_PRIMARY_MODEL: str = "qwen-max"
 AI_FALLBACK_MODEL: str = "gpt-4o-mini"

 # 阿里云OSS
 OSS_ACCESS_KEY_ID: str = ""
 OSS_ACCESS_KEY_SECRET: str = ""
 OSS_BUCKET_NAME: str = "shunshi-assets"
 OSS_ENDPOINT: str = "oss-cn-shanghai.aliyuncs.com"

 # 短信
 SMS_ACCESS_KEY_ID: str = ""
 SMS_ACCESS_KEY_SECRET: str = ""
```"""
    }
]
```

```

SMS_SIGN_NAME: str = "顺时"
SMS_TEMPLATE_CODE: str = ""

# 微信
WECHAT_APP_ID: str = ""
WECHAT_APP_SECRET: str = ""
WECHAT_MCH_ID: str = ""

# 极光推送
JPUSH_APP_KEY: str = ""
JPUSH_MASTER_SECRET: str = ""

# Meilisearch
MEILI_HOST: str = "http://localhost:7700"
MEILI_MASTER_KEY: str = ""

# App
APP_NAME: str = "顺时"
APP_ENV: str = "development"
APP_URL: str = "http://localhost:8000"

class Config:
    env_file = ".env"

settings = Settings()

```

输出完整文件，不省略。 “ “ “,{,” id” : “0.3” , “file” : “backend/app/database.py” , “prompt” : “ “ “创建 database.py, 包含: 1. SQLAlchemy异步引擎 (create_async_engine) 2. async_sessionmaker 3. Base declarative base 4. Redis连接 (使用redis.asyncio) 5. get_db异步生成器依赖 6. get_redis函数

使用config.py中的settings。完整文件，包含所有import。 “ “ “,{,” id” : “0.4” , “file” : “backend/app/core/exceptions.py” , “prompt” : “ “ “创建自定义异常系统:

class AppException(HTTPException): 基类，包含code和message class AuthenticationError(AppException): 401, “登录已过期，请重新登录” class PermissionDeniedError(AppException): 403, “抱歉，你没有权限访问” class NotFoundError(AppException): 404, “找不到你要的内容” class ValidationError(AppException): 422, “输入信息有误，请检查一下” class RateLimitError(AppException): 429, “请求太频繁了，休息一下吧” class PaymentError(AppException): 402, “支付处理失败，请重试” class AIServiceError(AppException): 503, “AI助手暂时忙碌，请稍后再试”

同时创建FastAPI exception_handler注册函数。完整文件。 “ “ “,{,” id” : “0.5” , “file” : “backend/app/core/response.py” , “prompt” : “ “ “创建统一响应函数:

1. def success_response(data=None, message= “success”) -> dict 返回 { “code” : 0, “message” : message, “data” : data}
2. def paginated_response(items, total, cursor=None, limit=20, has_more=False) -> dict 返回 { “code” : 0, “data” : { “items” : items, “total” : total, “cursor” : cursor, “limit” : limit, “has_more” : has_more}}
3. class ResponseModel(BaseModel): code, message, data (泛型Pydantic模型)

完整文件。 “ “ “,{,” # … 继续定义全部81个任务 … # 由于篇幅，这里展示核心框架，实际使用时需要填充完整任务列表]

===== 核心执行逻辑 =====

```
class AutoDeveloper: def init(self, api_key: str, model: str = "glm-4-plus" ): self.client = ZhipuAI(api_key=api_key)
self.model = model self.completed_files = [] self.project_root = PROJECT_ROOT
```

```
def get_context(self) -> str:
    """生成当前项目上下文"""
    files_str = "\n".join([f"- [{x}] {f}" for f in self.completed_files]) or "（尚无完成文件）"
    return PROJECT_CONTEXT.format(completed_files=files_str)

def call_ai(self, prompt: str, max_retries: int = 3) -> str:
    """调用AI API，支持重试"""
    full_prompt = f"""你是一位有10年经验的Python/FastAPI高级工程师。
```

请严格按照要求输出完整的代码文件。

```
{self.get_context()}
```

```
{prompt}
```

要求： 1. 输出完整可运行的代码，不要省略任何部分 2. 包含所有import语句 3. 包含完整的错误处理和类型注解 4. 只输出代码，不要解释性文字 5. 不要用markdown代码块包裹（直接输出纯代码） “ ”

```
for attempt in range(max_retries):
    try:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[{"role": "user", "content": full_prompt}],
            temperature=0.3, # 低温度=更确定性的代码
            max_tokens=4096,
        )
        content = response.choices[0].message.content

        # 清理markdown代码块
        if content.startswith("```"):
            lines = content.split("\n")
            content = "\n".join(lines[1:-1]) if lines[-1].strip() == "```" else "\n".join(lines[1:])

        return content.strip()
    except Exception as e:
        print(f" △ 第{attempt+1}次调用失败: {e}")
        if attempt < max_retries - 1:
            time.sleep(5)
        else:
            raise

def write_file(self, filepath: str, content: str):
    """写入文件"""
    full_path = self.project_root / filepath
    full_path.parent.mkdir(parents=True, exist_ok=True)
    full_path.write_text(content, encoding="utf-8")
    self.completed_files.append(filepath)
    print(f" ✓ 已创建: {filepath} ({len(content)} bytes)")

def validate_python(self, filepath: str) -> bool:
    """验证Python语法"""
    full_path = self.project_root / filepath
    if not filepath.endswith(".py"):
        return True
    try:
        compile(full_path.read_text(encoding="utf-8"), filepath, "exec")
        print(f" ✓ 语法验证通过")
        return True
    except SyntaxError as e:
        print(f" x 语法错误: {e}")
        return False

def fix_file(self, filepath: str, error: str) -> str:
    """让AI修复有问题的文件"""
    content = (self.project_root / filepath).read_text(encoding="utf-8")
    fix_prompt = f"""以下文件有错误，请修复并输出完整的修复后文件：
```

文件: {filepath} 错误: {error}

当前内容: {content}

请输出修复后的完整文件。 “ ” return self.call_ai(fix_prompt)

```
def execute_task(self, task: dict):
    """执行单个任务"""
    print(f"\n{'#' * 60}")
    print(f"任务 {task['id']}: {task['file']}")
    print(f"{'#' * 60}")

    content = self.call_ai(task["prompt"])
    self.write_file(task["file"], content)

    # 验证Python文件
    if task["file"].endswith(".py"):
        if not self.validate_python(task["file"]):
            print("  尝试自动修复...")
            fixed = self.fix_file(task["file"], "SyntaxError")
            self.write_file(task["file"], fixed)
            if not self.validate_python(task["file"]):
                print("  x 自动修复失败, 请手动检查")
                return False

    return True

def run(self, start_from: str = "0.1"):
    """执行全部任务"""
    print(f"\n{'#' * 60}")
    print(f"# 顺势项目自动化开发 - 开始")
    print(f"# 模型: {self.model}")
    print(f"# 起始任务: {start_from}")
    print(f"{'#' * 60}\n")

    started = False
    total = len(TASKS)

    for i, task in enumerate(TASKS):
        if task["id"] == start_from:
            started = True
        if not started:
            continue

        print(f"\n[{i+1}/{total}] ", end="")
        success = self.execute_task(task)

        if not success:
            print(f"\n△ 任务 {task['id']} 失败, 是否继续? (y/n): ", end="")
            if input().strip().lower() != 'y':
                print("已暂停。重新运行时使用 --start-from {task['id']} 从这里继续。")
                return

        # 每完成5个任务保存进度
        if (i + 1) % 5 == 0:
            self.save_progress()

        # API频率控制
        time.sleep(2)

    print(f"\n{'#' * 60}")
    print(f"# 全部任务完成! 共创建 {len(self.completed_files)} 个文件")
    print(f"{'#' * 60}")

def save_progress(self):
    """保存进度"""
    progress_file = self.project_root / ".dev_progress.json"
    progress_file.write_text(json.dumps({
        "completed_files": self.completed_files,
        "timestamp": time.strftime("%Y-%m-%d %H:%M:%S")
    }, ensure_ascii=False, indent=2), encoding="utf-8")
    print(f" 进度已保存 ({len(self.completed_files)} 个文件)")

def load_progress(self):
    """加载之前的进度"""
    progress_file = self.project_root / ".dev_progress.json"
    if progress_file.exists():
        data = json.loads(progress_file.read_text(encoding="utf-8"))
        self.completed_files = data.get("completed_files", [])
        print(f" 加载进度: 已完成 {len(self.completed_files)} 个文件")
```

===== 主入口 =====

```
if name == "main": parser = argparse.ArgumentParser(description= "顺时项目自动化开发" )
parser.add_argument( "-api-key" , required=True, help= "AI API Key" ) parser.add_argument( "-model" ,
default= "glm-4-plus" , help= "模型名称" ) parser.add_argument( "-start-from" , default= "0.1" , help= "从哪个任务
开始" ) parser.add_argument( "-resume" , action= "store_true" , help= "从上次中断处继续" ) args =
parser.parse_args()
```

```
dev = AutoDeveloper(api_key=args.api_key, model=args.model)

if args.resume:
    dev.load_progress()

dev.run(start_from=args.start_from)
```

```
---

## 方案三: Cursor Composer 一键方案

如果你使用 Cursor IDE, 可以使用 Composer 功能:

### 步骤

1. 在Cursor中打开空项目
2. 将 `顺时_终极开发文档.md` 放入项目根目录
3. 打开 Cursor Composer (Cmd+I)
4. 粘贴以下提示词:
```

@顺时_终极开发文档.md

请按照这份终极开发文档，从Phase 0开始逐步构建整个项目。

执行规则： 1. 严格按照文档中的数据库Schema创建Model 2. 严格按照文档中的API列表创建路由 3. 每创建一个文件就告诉我文件名 4. 代码必须完整，不要省略 5. 先完成后端（Phase 0-5），再完成前端（Phase 6-7）

请从Phase 0.1开始：创建 backend/requirements.txt

```
5. Cursor会自动创建文件，完成后你说"继续下一步"

### Cursor 特有优势
- Cursor可以同时看到项目中所有已创建的文件
- 自动维护上下文（不需要手动注入PROJECT_CONTEXT）
- 可以直接在IDE中运行和测试

---

## 方案对比

| 方案 | 工具 | 自动化程度 | 代码质量 | 适合场景 |
|-----|-----|-----|-----|-----|
| 方案一 | Claude Code | ***** | ***** | 最推荐, 全自动 |
| 方案二 | GLM API脚本 | ***** | ***** | GLM用户, 可自动化 |
| 方案三 | Cursor | *** | ***** | 半自动, 需手动继续 |
| 旧方式 | 直接丢文档 | * | * | 不推荐 |

---

## 关键成功因素

### 1. 低温度 = 高确定性
```python
temperature=0.3 # 写代码用低温度, 减少"创造性"
```

## 2. 单文件粒度

每次只要求AI输出一个完整文件。不要一次要求多个文件。

## 3. 先Schema后代码

确保数据库Schema、API接口定义是第一批完成的，后续所有代码都引用它们。

#### 4. 持续验证

每个文件创建后立即做语法检查。Python文件用compile(), TypeScript用tsc。

#### 5. 上下文注入

每次调用都注入：- 项目简介（2句话） - 代码规范（5条） - 已完成文件列表（自动更新）

这三样保证AI的输出前后一致。

---

文档版本：V1.0 用于：自动化驱动AI编程工具完成顺时/SEASONS项目开发